

Assessment-II : Real-Time Tower Health Monitoring in Telecom Industry Using Kafka

Business Problem

Our assessment focuses on the Telecommunication Industry and how they utilise data streaming in their everyday operations. Telecom companies like Jio, Airtel and Vodafone heavily depend on the network towers that they operate across India. Since these towers are responsible for transmitting vast amount of data every second, a sudden drop in their signal strength can lead to dropped calls, slow internet or service outage which can cause user inconvenience and may result in customer churn and loss of brand trust.

For our case, we have assumed a newly launched and rapidly growing Telecom company "**Bharat Connect**" that operates in limited cities in India. Being in their initial phase, Bharat Connect is facing trouble with tower signal drops and are receiving frequent complaints regarding the same. Thus, real-time monitoring of tower health becomes crucial for a new entrant in order to survive in the Telecom industry alongside big players like Jio and Airtel, that is why, for better user experience, Bharat Connect has decided actively respond to the towers experiencing signal drops, Bharat Connect has decided to implement a real-time surveillance system that continuously monitors tower signal health and triggers an alert to the company office whenever the signal drops below a certain threshold.

We have created a Kafka pipeline for data streaming of tower signal alert, following are the pipeline's components:

Producer

In real world scenario, the source for actual streaming data would be the network towers, transmitting vast amount of data every second. However, in our simulation, the data source is a `cdr_data.csv` file containing 1009 rows of information for all cities combined where Bharat Connect operates.

The Producer is responsible for pushing each row's data into the Kafka cluster, think of it as an on-field agent constantly reporting the status of the towers.

Broker

Here, the pushed data is stored temporarily, and it lets information go between those who send it (Producer) and those who receive it (Consumers). The broker receives information from hundreds of towers without choking. Even if one broker fails, the Kafka pipeline does not crash, rather the operations are transferred to another working broker by Zookeeper.

At Bharat Connect, the broker ensures that all tower health reports whether normal or critical, are never lost, never duplicated, and always delivered.

Topic

After the broker gets the data, it stores the data into a specific Topic to keep things organized, for example, in this case the topic is created for tower alert (`cdr_raw`). For example, Bharat Connect can also create various other Topics as well for related information like call logs, recharge patterns etc.

Partitions

Each Topic is further divided into smaller parts called Partitions for parallel data processing in order to increase scalability. Different consumers can read information from different partitions at the same time without any interference.

Though this component is a part of the Kafka pipeline, it has not been used in our simulation, we are only operating on a single partition for data consumption.

Stream and Storage

The tower health readings are continuously streamed and are stored in the MongoDB database as per the trigger. This allows Bharat Connect to do both real time inspection as well as analysis if historically stored data, crucial for their long-term efficiency.

Consumer/Consumer Groups

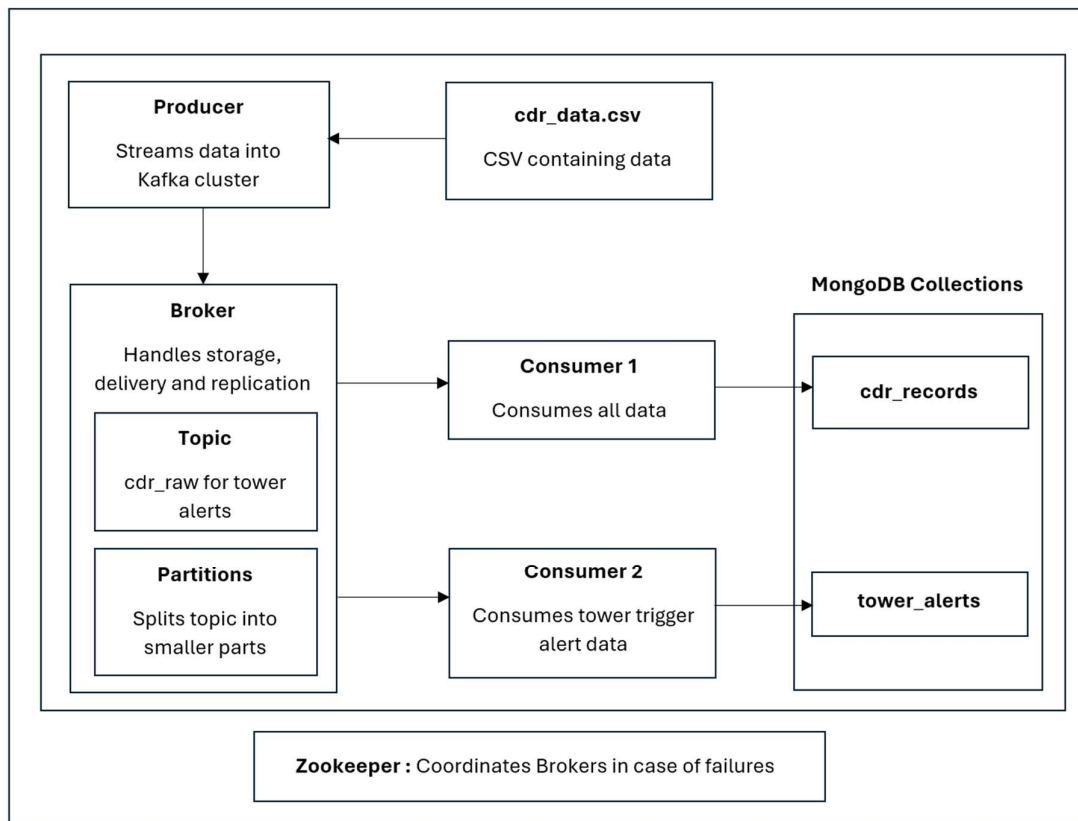
On the receiving end of this data stream is the Consumer that work like the network engineer continuously scanning the logs for tower alerts.

Our simulation has two Consumers, one that is responsible for consuming and pushing entire data from Kafka into MongoDB. Second consumer is trigger based, responsible for consuming only the relevant information (logs with tower_alert as event type) and pushing them to a separate MongoDB collection.

Zookeeper

Behind the scenes, Zookeeper works quietly, managing the entire Kafka pipeline. If a broker fails, it detects the failure immediately and switches the responsibility to another broker, thus ensuring active streaming.

Data Pipe Line- Flow Chart & Working



```
cd_r_producer.py X cd_r_consumer.py cd_r_consumer2.py
cd_r_producer.py > ...
1 import pandas as pd
2 import time
3 from kafka import KafkaProducer
4
5 # Load the CSV file
6 df = pd.read_csv('cdr_data.csv')
7
8 # Create Kafka producer
9 producer = KafkaProducer(bootstrap_servers='localhost:9092')
10
11 # Send each row to Kafka
12 for _, row in df.iterrows():
13     message = row.to_json()
14     producer.send('cdr_raw', message.encode('utf-8'))
15     print("Sent:", message)
16     time.sleep(1) # simulate real-time delay
17
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + - - - - -
```

```
PS C:\SDA> & C:/SDA/venv/Scripts/Activate.ps1
(venv) PS C:\SDA> & C:/SDA/venv/Scripts/python.exe c:/SDA/cdr_producer.py
Sent: {"timestamp": "24-07-2025 9:30", "event_type": "voice_call", "user_id": 6240.0, "location": "Ahmedabad", "device_type": "iOS", "signal_strength": "-85 dBm", "data_used_mb": 0.0, "voice_call_duration": 435, "recharge_amount": 0, "issue_flag": 0, "description": "Call out"}
Sent: {"timestamp": "24-07-2025 9:30", "event_type": "voice_call", "user_id": 2113.0, "location": "Delhi", "device_type": "Android", "signal_strength": "-61 dBm", "data_used_mb": 0.0, "voice_call_duration": 81, "recharge_amount": 0, "issue_flag": 0, "description": "Call out"}
Sent: {"timestamp": "24-07-2025 9:30", "event_type": "app_login", "user_id": 7010.0, "location": "Delhi", "device_type": "Android", "signal_strength": "-93 dBm", "data_used_mb": 0.0, "voice_call_duration": 0, "recharge_amount": 0, "issue_flag": 0, "description": "App login"}
Sent: {"timestamp": "24-07-2025 9:30", "event_type": "firmware_update", "user_id": 17371.0, "location": "Delhi", "device_type": "Router", "signal_strength": "-83 dBm", "data_used_mb": 383.69, "voice_call_duration": 0, "recharge_amount": 0, "issue_flag": 0, "description": "FW update"}
```

Producer

Streaming Telecom data into Kafka cluster

```
cd_r_producer.py X cd_r_consumer.py X cd_r_consumer2.py
cd_r_consumer.py > ...
12 print("X MongoDB Connection Failed:", e)
13 exit()
14
15 # Step 2: Connect to Kafka topic
16 try:
17     consumer = KafkaConsumer(
18         'cdr_raw',
19         bootstrap_servers='localhost:9092',
20         auto_offset_reset='earliest',
21         enable_auto_commit=True,
22         group_id='cdr-consumer-group'
23     )
24     print("Connected to Kafka topic 'cdr_raw'.\n")
25 except Exception as e:
26     print("X Kafka Connection Failed:", e)
27     exit()
28
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + - - - - -
```

```
Received: {"timestamp": "24-07-2025 10:33", "event_type": "sim_swap_detected", "user_id": 6139.0, "location": "Bengaluru", "device_type": "Android", "signal_strength": "-104 dBm", "data_used_mb": 0.0, "voice_call_duration": 0, "recharge_amount": 0, "issue_flag": 1, "description": "SIM swap"}
Inserted into MongoDB Atlas.

Received: {"timestamp": "24-07-2025 10:49", "event_type": "sim_swap_detected", "user_id": 1773.0, "location": "Jaipur", "device_type": "Android", "signal_strength": "-86 dBm", "data_used_mb": 0.0, "voice_call_duration": 0, "recharge_amount": 0, "issue_flag": 1, "description": "SIM swap"}
Inserted into MongoDB Atlas.

Received: {"timestamp": "24-07-2025 10:01", "event_type": "data_usage", "user_id": 3916.0, "location": "Ahmedabad", "device_type": "Android", "signal_strength": "-71 dBm", "data_used_mb": 288.22, "voice_call_duration": 0, "recharge_amount": 0, "issue_flag": 0, "description": "Video stream"}
Inserted into MongoDB Atlas.
```

Consumer-1

Receiving and sending all data into MongoDB

```
cd_r_producer.py X cd_r_consumer.py X cd_r_consumer2.py X
cd_r_consumer2.py > ...
14 except errors.DuplicateKeyError as e:
15     print("X Duplicate key error while creating index:", e)
16
17 # Step 3: Connect to Kafka topic
18 try:
19     consumer = KafkaConsumer(
20         'cdr_raw',
21         bootstrap_servers='localhost:9092',
22         value_deserializer=lambda m: json.loads(m.decode('utf-8')),
23         auto_offset_reset='earliest',
24         enable_auto_commit=True,
25         group_id='tower-alert-consumer'
26     )
27     print("Connected to Kafka topic 'cdr_raw'.")
28 except Exception as e:
29     print("X Kafka Connection Failed:", e)
30     exit()
31
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + - - - - -
```

```
Microsoft Windows [Version 10.0.26100.4770]
(c) Microsoft Corporation. All rights reserved.

C:\SDA>C:/SDA/venv/Scripts/activate.bat
(venv) C:\SDA>python cdr_consumer2.py
Unique index ensured on call_id.
Connected to Kafka topic 'cdr_raw'.
Skipped non-Tower alert record.
Skipped non-Tower alert record.
Skipped non-Tower alert record.
Skipped non-Tower alert record.
Skipped non-Tower alert record.
Tower Alert Received: {"timestamp": "24-07-2025 9:30", "event_type": "tower_alert", "user_id": None, "location": "Hyderabad", "device_type": "Tower", "signal_strength": "-94 dBm", "data_used_mb": 0.0, "voice_call_duration": 0, "recharge_amount": 0, "issue_flag": 1, "description": "Tower alert"}
Inserted into MongoDB Atlas.
```

Consumer-2

Receiving and sending triggered data into MongoDB

telecom_db				
cluster0.kbix.mongodb.net > telecom_db				
Sort by Collection Name				
View				
cdr_records				
Storage size:	Documents:	Avg. document size:	Indexes:	Total index size:
69.63 kB	1K	284.00 B	1	65.54 kB
tower_alerts				
Storage size:	Documents:	Avg. document size:	Indexes:	Total index size:
20.48 kB	109	276.00 B	1	36.86 kB

Database

MongoDB with two separate collections.